

Cairo University — Faculty of Computers and Artificial Intelligence

Supervised Learning — Spring 2026

Course Project

Instructor: Dr. Ghada Dahy | Eng. Hamza EmadElDin, Eng. Abdelrahman Sayed, Eng. Sherif Magdy

Autonomous Vehicle Perception Module

Name	ID
Abdullah Ali	20230227
Mohammed Ashraf	20230319
Ayman Sayed	20230081
Islam waleed	20230062
Ali Raouf	20220212

1. Project Overview

This project presents a comprehensive comparative study of traffic sign classification using the GTSRB (German Traffic Sign Recognition Benchmark) dataset and BDD100k temporal dataset. Multiple approaches are implemented and evaluated across three phases: traditional machine learning with hand-crafted features, deep neural networks, and temporal/sequential models.

Key Statistics

Primary Dataset	GTSRB (German Traffic Sign Recognition Benchmark)
Traffic Sign Classes	43 classes
Training Images	~39,209 images
Test Images	~12,630 images
Temporal Dataset	BDD100k — 4,000 frames (2,500 train / 500 val / 1,000 test)

Study Phases

- Phase 1: Classical computer vision with HOG features and traditional ML classifiers
- Phase 2: Deep neural networks including custom CNNs, transfer learning, and autoencoders
- Phase 3: Temporal/sequential models (RNN, LSTM, GRU, Transformer) on BDD100k

2. Phase 1: Traditional Machine Learning

2.1 Methodology

Feature Extraction: HOG (Histogram of Oriented Gradients)

HOG captures edge and texture patterns in traffic signs effectively. Configuration used:

- Orientations: 8
- Pixels per cell: 8x8
- Cells per block: 1x1
- Feature vector dimension: 64 (after extraction)

Dimensionality Reduction: PCA

PCA was applied to compress HOG features while retaining classification power. The table below summarizes the trade-off analysis:

Config	Features	Val Accuracy	Test Accuracy	Compression
32 components	32	0.9250	0.6023	93.75%
64 components	64	0.9495	0.6474	87.50%
128 components*	128	0.9603	0.6846	75.00%
256 components	256	0.9626	0.7017	50.00%
Raw (No PCA)	512	0.9580	0.6973	None

* Selected configuration: 128 PCA components — optimal balance between speed and accuracy.

2.2 Models Trained

Model 1: Naive Bayes (Baseline)

- Type: Probabilistic classifier (Gaussian NB)
- Tuning: var_smoothing via GridSearchCV
- Best var_smoothing: 1.873817422860383e-05

Metric	Validation	Test
Accuracy	0.8460	~0.7451 (raw)
AUC-ROC	0.9948	0.9849

Model 2: K-Nearest Neighbors

- Type: Instance-based classifier
- Best config: k=3, Euclidean distance
- Tuning: n_neighbors = 3-10, metrics = {euclidean, manhattan}

Metric	Validation	Test
Accuracy	0.9651	0.6973 (raw)
AUC-ROC	0.9956	0.9019

Model 3: Random Forest

- Type: Ensemble learning (100 decision trees)
- Configuration: Default scikit-learn settings on PCA-128 features

Metric	Validation	Test
Accuracy	0.8674	0.7086
AUC-ROC	0.9951	0.9730

Model 4: XGBoost

- Type: Gradient boosting
- Config: 100 estimators, depth=6, learning_rate=0.1

Metric	Validation	Test
Accuracy	0.8884	0.7683
AUC-ROC	0.9977	0.9883

2.3 Phase 1 Summary

XGBoost achieved the best test accuracy among traditional models at 76.83%, with strong AUC-ROC of 0.9883. KNN showed high validation accuracy (96.51%) but suffered from overfitting on the test set. All models used PCA-128 features for the final evaluation.

3. Phase 2: Deep Learning

3.1 Model 1: SimpleCNN (Custom 2-Layer CNN)

Architecture

Input (64x64, RGB) => Conv2d(3->16, 3x3) + ReLU + MaxPool(2x2) => Conv2d(16->32, 3x3) + ReLU + MaxPool(2x2) => Flatten => FC(32x16x16->128) + ReLU => FC(128->43) [Output]

Training Configuration

- Epochs: 5
- Batch Size: 64
- Optimizers evaluated: Adam, SGD
- Schedulers evaluated: StepLR, CosineAnnealing

Results

Optimizer	Scheduler	Learning Rate	Val Accuracy	Test Accuracy
Adam	StepLR	1e-3	0.9807	0.9028
SGD	CosineAnnealing	5e-3	0.9800	0.8904

3.2 Model 2: Transfer Learning (Head-First Fine-Tuning)

Backbones Evaluated

- MobileNetV2 — Lightweight (~3.5M parameters, 14 MB)
- VGG16 — High accuracy (~138M parameters, 528 MB)

Two-Phase Training Strategy

Phase 1 — Head Training (2 epochs): Freeze entire backbone; train classifier head only at LR=1e-3 to stabilize the new output layer.

Phase 2 — Fine-tuning (3 epochs): Unfreeze top layers (4 for MobileNet, 10 for VGG16) plus classifier; train at LR=1e-4 to adapt pretrained features to traffic signs.

Why Head-First?

- Prevents degradation of pretrained features
- Noisy random-head gradients won't corrupt backbone early
- Mimics curriculum learning: easy to hard

Results

Base Model	Val Accuracy	Test Accuracy	Train Time (s)	Model Size
MobileNetV2	0.9852	0.9081	465.4	14 MB
VGG16	0.9967	0.9485	710.1	528 MB

3.3 Model 3: Denoising Autoencoder

Purpose

Learn robust, noise-invariant feature representations and reconstruct corrupted traffic sign images.

Architecture

Encoder: Conv(3->32)+BN+ReLU+MaxPool => Conv(32->64)+BN+ReLU+MaxPool => Conv(64->128)+BN+ReLU => Bottleneck [128x16x16]

Decoder: ConvTranspose(128->64)+BN+ReLU => ConvTranspose(64->32)+BN+ReLU => Conv(32->3)+Sigmoid => Output (64x64, RGB)

Training Configuration

- Epochs: 15 | Batch Size: 64
- Noise Factor: 0.1 (10% Gaussian noise injected during training)
- Optimizer: Adam (LR=5e-4)
- Loss: L1 (MAE) for robustness to outliers

Reconstruction Performance

Split	MAE	MSE
Validation (best epoch)	0.0210	0.0010
Test	0.0209	0.0010

3.4 Phase 2 Summary

VGG16 with head-first fine-tuning achieved the highest test accuracy at 94.85%, significantly outperforming all traditional ML approaches. MobileNetV2 offers an excellent efficiency-accuracy trade-off (90.81% accuracy at only 14 MB and ~35% less training time) making it suitable for edge deployment.

4. Phase 3: Temporal Models

4.1 Dataset: BDD100k

A 2-frame sequence dataset with multi-label classification for autonomous driving scenarios. Each frame is annotated with up to 3 labels.

Task	Multi-label classification (Green Light / Red Light / Road Sign)
Training	1,360 sequences 2,500 frames
Validation	276 sequences 500 frames
Testing	555 sequences 1,000 frames

Label Distribution (Training Set — 2,500 frames)

Label	Training	Validation
Red Light present	19.9%	32.0%
Green Light present	26.2%	29.0%
Road Sign present	58.8%	64.6%

4.2 Model 1: RNN

Label	Accuracy
Red Light	0.8404
Green Light	0.8652
Road Sign	0.6562
Average	0.7873

4.3 Model 2: LSTM

Label	Accuracy
Red Light	0.8315
Green Light	0.8337

Road Sign	0.6809
Average	0.7820

4.4 Model 3: GRU

Label	Accuracy
Red Light	0.8292
Green Light	0.6787
Road Sign	0.6719
Average	0.7266

4.5 Model 4: Transformer

Label	Accuracy
Red Light	0.6584
Green Light	0.7461
Road Sign	0.6719
Average	0.6921

5. Conclusion & Key Findings

5.1 Technical Stack

- Languages & Frameworks: Python 3.8+, PyTorch 2.0+, TorchVision, scikit-learn, XGBoost
- Hardware: NVIDIA T4 / Colab GPU | 16 GB RAM | ~7-9 hours total training
- Optimizations: CUDA acceleration, mixed precision (torch.autocast + GradScaler), multi-worker DataLoaders

5.2 Work Done

Person	Work Done
Abdullah Ali	Phase 1 Data preprocessing, FT and Phase Optimizers Phase 3 Data Loader and FT
Ayman Sayed	PCA, KNN, NB Feature map visualization RNN
Mohammed Ashraf	Ensemble Learning Transfer Learning GRU
Islam Waleed	Hyperparameter optimization CNN LSTM
Ali Kelany	Auto Encoder Transformers

5.3 Conclusion

This project explored a full pipeline for traffic sign and driving-scene perception by comparing classical machine learning, deep learning, and temporal sequence modeling. In Phase 1, HOG features with PCA (128 components) enabled efficient training, with XGBoost achieving the strongest traditional performance (76.83% test accuracy, AUC-ROC 0.9883). In Phase 2, deep models substantially improved recognition, where transfer learning delivered the best results—VGG16 reached 94.85% test accuracy, while MobileNetV2 provided a practical edge-deployment option (90.81% accuracy at 14 MB). In Phase 3, temporal models on BDD100k demonstrated the value of sequence-aware learning for multi-label driving cues, with RNN/LSTM/GRU outperforming the Transformer baseline on average accuracy for this 2-frame setup. Overall, the study confirms that modern pretrained CNN backbones provide the largest accuracy gains for sign recognition, while temporal modeling remains essential when decisions depend on consecutive frames in real driving scenarios.

Authors: Abdullah Ali | Mohammed Ashraf | Ayman Sayed | Islam Waleed | Ali Kelany